

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: **ARTIFICIAL** **COURSE CODE:** **MLA010**
 INTELLIGENCE **AND** **1**
 EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)*
Assessment Tool Weightage: 40%

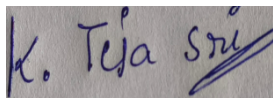
Total Marks: 100
No. of Questions: 1

Time: 90 Mins

Annexure A Expert System Development Project

Code of Conduct:

I K.Teja Sri (192525249) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System:

	Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.
--	---

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

		backtracking effectively.			
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

STUDENT ACADEMIC ADVISORY EXPERT SYSTEM USING PROLOG

Problem Analysis and Domain Understanding

1.1 Introduction

Academic performance depends on several factors such as attendance, internal marks, assignment performance, learning difficulties, previous academic performance, and study habits. Students who experience problems in one or more of these areas may require suitable academic support.

The **Student Academic Advisory Expert System** is a rule-based expert system developed using **SWI-Prolog**. The system analyzes student academic information and generates suitable recommendations using production rules.

The system uses facts such as:

- Attendance status
- Internal examination marks
- Assignment performance
- Learning difficulty
- Previous academic performance
- Study habits

Based on these facts, the system recommends suitable interventions such as:

- Improving attendance
- Attending remedial classes
- Improving assignment performance
- Taking faculty guidance
- Following regular study habits

1.2 Problem Statement

To develop a rule-based expert system that provides academic recommendations based on student performance and learning-related conditions using **Prolog facts, production rules, forward chaining, and backward chaining**.

1.3 Objectives

The main objectives are:

1. To represent student academic information using Prolog facts.

2. To develop production rules using IF–THEN logic.
3. To implement academic recommendations using Prolog.
4. To demonstrate forward chaining.
5. To demonstrate backward chaining.
6. To demonstrate unification and backtracking.
7. To compare procedural and non-procedural approaches.
8. To test the accuracy of the expert system.

2. Knowledge Acquisition

Knowledge for the system is represented using basic academic conditions and corresponding recommendations.

Academic Condition	Possible Recommendation
Low attendance	Improve attendance
Low internal marks	Attend remedial classes
Poor assignments	Improve assignments
Learning difficulty	Faculty guidance
Weak previous performance + poor study habit	Regular study

Knowledge Sources

The knowledge can be obtained from:

- General academic advisory practices
- Institutional academic guidelines
- Faculty recommendations
- Student performance evaluation criteria

The collected knowledge is converted into logical facts and production rules suitable for Prolog.

3. Knowledge Representation

The system represents knowledge using **facts and rules**.

3.1 Student Fact

`student(teja).`

This represents that Teja is a student.

3.2 Academic Facts

attendance(teja, low).
internal_marks(teja, low).
assignment(teja, poor).
learning_difficulty(teja, yes).
previous_performance(teja, weak).
study_habit(teja, poor).

These facts represent the academic conditions of the student.

3.3 Production Rules

The knowledge is represented through rules.

recommend(S, improve_attendance) :-
 attendance(S, low).

Meaning:

IF attendance is low THEN recommend improving attendance.

Another example:

recommend(S, remedial_classes) :-
 internal_marks(S, low).

Meaning:

IF internal marks are low THEN recommend remedial classes.

Knowledge Representation Table:

Fact/Condition	Rule	Conclusion
Attendance is low	Low attendance → improvement required	Improve attendance
Internal marks are low	Low marks → remedial support	Remedial classes
Assignment is poor	Poor assignment → improvement	Improve assignments
Learning difficulty exists	Difficulty → guidance	Faculty guidance
Weak performance + poor study habit	Both conditions → study recommendation	Regular study

4. Production Rule / Knowledge Base Design

The production-rule knowledge base contains the following rules:

Rule 1 – Attendance

```
recommend(S, improve_attendance) :-  
    attendance(S, low).
```

Rule 2 – Internal Marks

```
recommend(S, remedial_classes) :-  
    internal_marks(S, low).
```

Rule 3 – Assignment

```
recommend(S, improve_assignments) :-  
    assignment(S, poor).
```

Rule 4 – Learning Difficulty

```
recommend(S, faculty_guidance) :-  
    learning_difficulty(S, yes).
```

Rule 5 – Study Habit

```
recommend(S, regular_study) :-  
    previous_performance(S, weak),  
    study_habit(S, poor).
```

These rules allow the expert system to derive recommendations from the available student facts.

5. Prolog Implementation and Programming

The system is implemented using **SWI-Prolog**.

The main Prolog concepts used are:

- Facts
- Rules
- Predicates
- Variables
- Unification
- Backtracking

- Logical inference

Main predicates

student/1
attendance/2
internal_marks/2
assignment/2
learning_difficulty/2
previous_performance/2
study_habit/2
recommend/2
academic_advice/2
forward_chain/2
backward_demo/2
show_advice/1

Implementation Procedure

1. Create academic_advisor.pl.
2. Enter the facts and production rules.
3. Open SWI-Prolog.
4. Consult the .pl file.
5. Execute queries.
6. Observe the generated recommendations.
7. Test forward and backward reasoning.

```

===== STUDENT ACADEMIC ADVISORY EXPERT SYSTEM =====
===== STUDENT FACTS =====
===== PRODUCTION RULES =====
===== RECOMMENDED COURSE =====
===== FORWARD CHAINING =====
===== DISPLAY FORWARD RESULTS =====
===== COMPLETE ACADEMIC ADVICE =====
===== SYSTEM DISPLAY =====

```

Output:

```
38 ?- consult(['c:/Users/Rama Krishna K/Desktop/Student_Academic_Advisory/academic_advisor.pl']).
true.
```

6. Forward Chaining Implementation and Demonstration

6.1 Forward Chaining

Forward chaining is a **data-driven reasoning technique**. It starts with known facts and applies applicable rules to derive conclusions.

For this system:

Student Facts



Check Conditions



Match Production Rules

↓
Apply Applicable Rules
↓
Generate Recommendations

Example

Initial facts:

attendance = low
internal marks = low
assignment = poor
learning difficulty = yes
previous performance = weak
study habit = poor

The system applies the rules and derives:

improve_attendance
remedial_classes
improve_assignments
faculty_guidance
regular_study

Forward Chaining Query

Run:

?- show_forward(teja).

output:

```
44 ?- forward_chain(teja, X).
X = [improve_attendance, remedial_classes, improve_assignments, faculty_guidance, regular_study].

45 ?- show_forward(teja).
FORWARD CHAINING RESULTS
Student: teja
Recommendations:
- improve_attendance
- remedial_classes
- improve_assignments
- faculty_guidance
- regular_study
true.
```

7. Backward Chaining Implementation and Demonstration

7.1 Backward Chaining

Backward chaining is a **goal-driven reasoning technique**. It starts with a desired conclusion and works backward to determine whether the required facts are available.

For example, consider the goal:

```
?- recommend(teja, remedial_classes).
```

Prolog searches for a rule that can prove this recommendation.

It finds:

```
recommend(S, remedial_classes) :-  
    internal_marks(S, low).
```

It then checks:

```
internal_marks(teja, low).
```

Since this fact exists, the query succeeds.

Output

true.

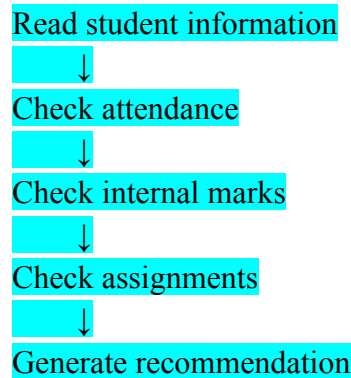
```
41 ?- recommend(teja, improve_attendance).  
true.  
  
42 ?- recommend(teja, X).  
X = improve_attendance ;  
X = remedial_classes ;  
X = improve_assignments ;  
X = faculty_guidance ;  
X = regular_study.  
  
43 ?- backward_demo(teja, remedial_classes).  
true.
```

8. Procedural and Non-Procedural Paradigm Analysis

8.1 Procedural Approach

A procedural approach specifies **how** the solution should be obtained through a sequence of instructions.

For example:



Languages such as C, C++, and Java commonly use procedural or algorithmic instructions.

8.2 Non-Procedural / Declarative Approach

Prolog is mainly a **declarative programming language**.

Instead of specifying every step, we describe:

- Facts
- Relationships
- Rules
- Conditions

Prolog's inference mechanism determines how the conclusion is obtained.

Comparison:

Procedural	Non-Procedural / Declarative
Specifies how to solve the problem	Specifies what is true
Uses explicit sequence of operations	Uses facts and logical rules
Programmer controls execution	Prolog performs inference
Common in C/C++/Java	Prolog is a major example
Algorithm-oriented	Knowledge-oriented

Conclusion:

The Student Academic Advisory System is suitable for a **non-procedural/declarative approach** because academic knowledge can naturally be represented as facts and rules.

9. Unification, Backtracking and Inference Accuracy

9.1 Unification

Unification is the process of matching terms and assigning values to variables.

For example:

```
?- recommend(teja, X).
```

Prolog can unify X with:

X = improve_attendance

then:

X = remedial_classes

and so on.

9.2 Backtracking

Backtracking allows Prolog to search for alternative solutions when multiple rules can satisfy a query.

Query

```
?- recommend(teja, X).
```

Result:.

```
41 ?- recommend(teja, improve_attendance).  
true.
```

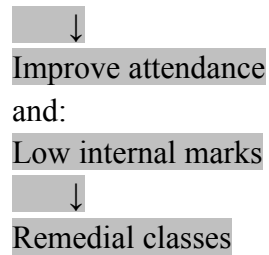
```
42 ?- recommend(teja, X).  
X = improve_attendance ;  
X = remedial_classes ;  
X = improve_assignments ;  
X = faculty_guidance ;  
X = regular_study.
```

9.3 Inference Accuracy

The system produces recommendations according to the predefined facts and rules. For

example:

```
Low attendance
```



The conclusions are consistent with the knowledge encoded in the system.

10. Testing, Results and System Demonstration

The system is tested using different queries.

Test Case 1 – Student Verification

Query:

?- student(teja).

Expected result:

true.

Test Case 2 – Attendance Recommendation

Query:

?- recommend(teja, improve_attendance).

Expected result:

true.

Test Case 3 – Remedial Classes

Query:

?- recommend(teja, remedial_classes).

Expected result:

true.

Test Case 4 – All Recommendations

Query:

?- recommend(teja, X).

Expected result:

X = improve_attendance ;

X = remedial_classes ;

X = improve_assignments ;

X = faculty_guidance ;

X = regular_study.

Test Case 5 – Complete System

Query:

```
?- show_advice(teja).
```

Output:

```
46 ?- show_advice(teja).  
=====
```

STUDENT ACADEMIC ADVISORY SYSTEM

```
=====
```

Student: teja

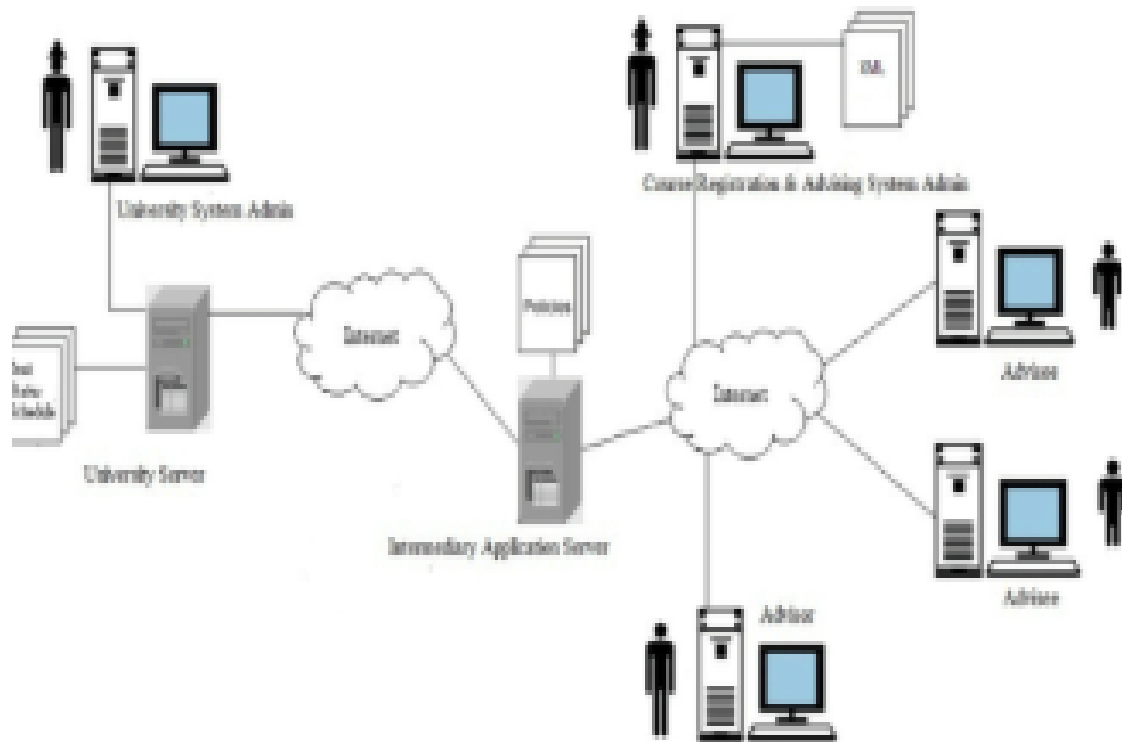
Recommendations:

- improve_attendance
- remedial_classes
- improve_assignments
- faculty_guidance
- regular_study

true.

11. System Architecture

The overall system can be represented as:



Explanation

The student information is stored as facts. The knowledge base contains production rules. The inference engine processes the facts and rules using forward or backward reasoning. Finally, the system generates appropriate academic recommendations.

Figure: Architecture of Student Academic Advisory Expert System

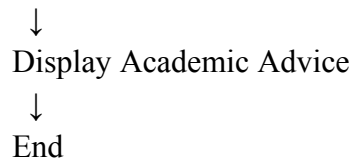
12. Overall System Working

The working of the system can be summarized as follows:

```

Start
↓
Enter/Store Student Facts
↓
Load Knowledge Base
↓
Apply Production Rules
↓
Perform Inference
↓
Generate Recommendations

```



The system uses logical reasoning rather than a statistical machine-learning model. Therefore, the recommendations are directly explainable from the facts and rules.

13. Advantages

1. Simple and easy to understand.
2. Uses explainable production rules.
3. Provides consistent recommendations.
4. Demonstrates logical inference.
5. Supports unification and backtracking.
6. Can demonstrate both forward and backward reasoning.
7. Easy to modify by adding new rules.
8. Suitable for academic advisory applications.

14. Limitations

1. The system depends on predefined facts and rules.
2. It does not automatically learn from new student data.
3. Recommendations are limited to the knowledge base.
4. Complex academic situations may require additional rules.
5. It is intended as an advisory system and not as a replacement for faculty judgment.

15. Future Scope

The system can be enhanced by:

- Adding multiple students.
- Adding more academic parameters.
- Including course-specific recommendations.
- Adding a graphical user interface.
- Connecting the system with a student database.
- Adding more detailed explanation facilities.
- Integrating historical academic data.
- Extending the rule base for different academic situations.

16. Conclusion

The **Student Academic Advisory Expert System** successfully demonstrates the development of a rule-based expert system using **SWI-Prolog**. Student academic information is represented using facts, while production rules are used to derive appropriate recommendations.

The system demonstrates important AI and expert-system concepts including **knowledge representation, production rules, forward chaining, backward chaining, unification, backtracking, and logical inference**.

The project also demonstrates the difference between procedural and declarative programming paradigms. Prolog provides a suitable declarative environment because the academic knowledge can be represented naturally using facts and logical rules.

17. References

You can include references such as:

1. Stuart Russell and Peter Norvig, **Artificial Intelligence: A Modern Approach**, Pearson.
2. Ivan Bratko, **Prolog Programming for Artificial Intelligence**, Pearson.
3. W. F. Clocksin and C. S. Mellish, **Programming in Prolog**, Springer.
4. SWI-Prolog Documentation – Prolog language and reference documentation.
5. George F. Luger, **Artificial Intelligence: Structures and Strategies for Complex Problem Solving**, Pearson.